



CRUD APPLICATO

Programmazione

Che cos'è un CRUD (Create, Read, Update, Delete)

Un CRUD è un insieme di quattro operazioni fondamentali utilizzate in qualsiasi sistema informatico che gestisce dati persistenti.

L'acronimo identifica i quattro comportamenti minimi che un'applicazione deve offrire per interagire con un archivio dati: Create (creare nuovi record), Read (leggere o recuperare dati), Update (aggiornare o modificare dati esistenti), e Delete (eliminare dati).

Queste quattro azioni costituiscono il ciclo base della manipolazione delle informazioni ed è il motivo per cui quasi ogni software gestito tramite database o API segue questo schema. In altre parole, un CRUD rappresenta la “grammatica” con cui un sistema tratta i dati.



Come si usa nel contesto IT e dei database

Nel mondo IT, un CRUD viene implementato tipicamente come insieme di endpoint API, funzioni applicative o query SQL che permettono a un'applicazione di interagire con un database.

Per esempio, in SQL, il mapping naturale del CRUD è: INSERT → Create, SELECT → Read, UPDATE → Update, DELETE → Delete.

Queste operazioni costituiscono la base di qualunque servizio che memorizza informazioni, dai sistemi gestionali ai microservizi basati su REST.

A livello architetturale, il CRUD determina la struttura dei controller, la progettazione delle interfacce utenti, e persino gli schemi di persistenza: ogni tabella o entità del dominio avrà generalmente un set di operazioni CRUD associato, che diventa la base su cui costruire logiche più complesse come validazioni, controlli di accesso o automazioni lato backend.



Installare MySQL Server sulla macchina locale o sul server.

Avviare il servizio MySQL, verificando che il database server sia in esecuzione.

Accedere a MySQL tramite terminale o tool grafico, ad esempio MySQL Workbench.

Creare un database con un comando SQL:

1. CREATE DATABASE nome_database;

Selezionare il database da utilizzare:

1. USE nome_database;

Creare le tabelle definendo colonne, tipi di dato e chiavi:

**1. CREATE TABLE utenti (
2. id INT AUTO_INCREMENT PRIMARY KEY,
3. nome VARCHAR(50),
4. email VARCHAR(100)
5.);**



CREATE – Creare un nuovo dato

L'operazione Create serve ad aggiungere un nuovo record al sistema. In un vero backend potresti fare una chiamata POST verso un'API; in un database useresti INSERT.

In JavaScript, se simuli un archivio dati con un array, creare significa semplicemente aggiungere un oggetto alla lista.

L'importante è garantire che il nuovo elemento abbia un'identità unica (es. un id), dato che servirà per tutte le altre operazioni CRUD.



CREATE – Creare un nuovo dato

```
1. let utenti = [];  
2.  
3. // CREATE  
4. function createUser(nome) {  
5.   const nuovoUtente = {  
6.     id: Date.now(), // simuliamo un ID unico  
7.     nome: nome  
8.   };  
9.  
10.  utenti.push(nuovoUtente);  
11.  return nuovoUtente;  
12.}  
13.  
14. console.log(createUser("Mirko"));  
15.
```



READ – Leggere o recuperare dati

L'operazione Read serve a ottenere dati dal sistema. In un DB è l'equivalente di una SELECT, mentre in un'applicazione web è spesso una chiamata GET.

Nel nostro caso, leggere significa semplicemente restituire l'intera lista o cercare un singolo elemento tramite una condizione (es. “trova utente con id X”).

L'operazione Read non modifica nulla: è puramente consultiva.



READ – Leggere o recuperare dati

```
1.// READ - ottenere tutti gli utenti
2.function getAllUsers() {
3.  return utenti;
4.}
5.
6.// READ - trovare un utente per ID
7.function getUserById(id) {
8.  return utenti.find(u => u.id === id);
9.}
10.
11.console.log(getAllUsers());
12.
```



UPDATE – Aggiornare dati esistenti

L'operazione Update modifica un record già presente senza crearne uno nuovo.

In un contesto SQL è equivalente alla UPDATE, mentre in un API sarebbe una chiamata PUT o PATCH.

La logica fondamentale è:

- 1. cercare il dato da modificare (es. tramite ID),**
- 2. cambiare solo i campi desiderati,**
- 3. salvare nuovamente l'oggetto aggiornato.**



UPDATE – Aggiornare dati esistenti

```
1.// UPDATE
2.function updateUser(id, nuovoNome) {
3.  const utente = utenti.find(u => u.id === id);
4.  if (!utente) return null;
5.
6.  utente.nome = nuovoNome;
7.  return utente;
8.}
9.
10.// esempio di utilizzo:
11.const idDaAggiornare = utenti[0].id;
12.console.log(updateUser(idDaAggiornare, "Mirko Updated"));
13.
```



DELETE – Eliminare un dato

La Delete rimuove un record dal sistema.

Nel mondo SQL è una DELETE, mentre nelle API è spesso una chiamata DELETE verso un endpoint che rappresenta la risorsa.

In JavaScript eliminare un elemento da una lista significa filtrarlo oppure rimuoverlo tramite indice.

È importante restituire un valore che indichi se la cancellazione è avvenuta oppure no.



DELETE – Eliminare un dato

```

1. // DELETE
2. function deleteUser(id) {
3.   const lunghezzaOriginale = utenti.length;
4.   utenti = utenti.filter(u => u.id !== id);
5.
6.   return utenti.length < lunghezzaOriginale; // true se eliminato
7. }
8.
9. // esempio di utilizzo:
10. console.log(deleteUser(idDaAggiornare));
11.
12. // true se l'utente è stato cancellato
13.

```



Server.js

```
1.const express = require("express");
2.const db = require("./db");
3.
4.const app = express();
5.
6.app.use(express.json());
7.
8.// CREATE - inserire uno studente
9.app.post("/studenti", (req, res) => {
10.  const { nome, cognome, email } = req.body;
11.
12.  const sql = "INSERT INTO studenti (nome, cognome, email) VALUES (?, ?, ?)";
13.
14.  db.query(sql, [nome, cognome, email], (errore, risultato) => {
15.    if (errore) {
16.      res.status(500).json({ messaggio: "Errore inserimento studente" });
17.    } else {
18.      res.json({
19.        messaggio: "Studente inserito correttamente",
20.        id: risultato.insertId
21.      });
22.    }
23.  });
24.});
25.
26.// READ - leggere tutti gli studenti
27.app.get("/studenti", (req, res) => {
28.  const sql = "SELECT * FROM studenti";
29.
30.  db.query(sql, (errore, risultati) => {
31.    if (errore) {
32.      res.status(500).json({ messaggio: "Errore lettura studenti" });
33.    } else {
34.      res.json(risultati);
35.    }
36.  });
37.});
38.
39.// READ - leggere uno studente per id
40.app.get("/studenti/id", (req, res) => {
41.  const id = req.params.id;
42.
43.  const sql = "SELECT * FROM studenti WHERE id = ?";
44.
45.  db.query(sql, [id], (errore, risultati) => {
46.    if (errore) {
47.      res.status(500).json({ messaggio: "Errore lettura studente" });
48.    } else if (risultati.length === 0) {
49.      res.status(404).json({ messaggio: "Studente non trovato" });
50.    } else {
51.      res.json(risultati[0]);
52.    }
53.  });
54.});
55.
56.// UPDATE - modificare uno studente
57.app.put("/studenti/id", (req, res) => {
58.  const id = req.params.id;
59.  const { nome, cognome, email } = req.body;
60.
61.  const sql = "UPDATE studenti SET nome = ?, cognome = ?, email = ? WHERE id = ?";
62.
63.  db.query(sql, [nome, cognome, email, id], (errore, risultato) => {
64.    if (errore) {
65.      res.status(500).json({ messaggio: "Errore modifica studente" });
66.    } else if (risultato.affectedRows === 0) {
67.      res.status(404).json({ messaggio: "Studente non trovato" });
68.    } else {
69.      res.json({ messaggio: "Studente modificato correttamente" });
70.    }
71.  });
72.});
73.
74.// DELETE - eliminare uno studente
75.app.delete("/studenti/id", (req, res) => {
76.  const id = req.params.id;
77.
78.  const sql = "DELETE FROM studenti WHERE id = ?";
79.
80.  db.query(sql, [id], (errore, risultato) => {
81.    if (errore) {
82.      res.status(500).json({ messaggio: "Errore eliminazione studente" });
83.    } else if (risultato.affectedRows === 0) {
84.      res.status(404).json({ messaggio: "Studente non trovato" });
85.    } else {
86.      res.json({ messaggio: "Studente eliminato correttamente" });
87.    }
88.  });
89.});
90.
91.app.listen(3000, () => {
92.  console.log("Server avviato su http://localhost:3000");
93.});
```



File db.js:

```
1.const mysql = require("mysql2");
2.
3.const db = mysql.createConnection({
4.  host: "localhost",
5.  user: "root",
6.  password: "",
7.  database: "scuola_db"
8.});
9.
10.db.connect((errore) => {
11.  if (errore) {
12.    console.log("Errore connessione database:", errore);
13.  } else {
14.    console.log("Connessione a MySQL riuscita");
15.  }
16.});
17.
18.module.exports = db;
```

Database MySQL:

```
1.CREATE DATABASE scuola_db;
2.
3.USE scuola_db;
4.
5.CREATE TABLE studenti (
6.  id INT AUTO_INCREMENT PRIMARY KEY,
7.  nome VARCHAR(50) NOT NULL,
8.  cognome VARCHAR(50) NOT NULL,
9.  email VARCHAR(100) NOT NULL
10.);
```

Esempi di chiamate

Inserimento

```
1.POST http://localhost:3000/studenti
2.
3.{
4.  "nome": "Luca",
5.  "cognome": "Rossi",
6.  "email": "luca.rossi@email.com"
7.}
```

Modifica

```
1.PUT http://localhost:3000/studenti/1
2.
3.{
4.  "nome": "Luca",
5.  "cognome": "Bianchi",
6.  "email": "luca.bianchi@email.com"
7.}
```


Buon DAVANTE a tutti

